BHOOMI ANNAMDEVARA

24BCI0130

COMPILER DESIGN LAB

Lab Assignment - 01

Design of a Symbol Table Manager

Aim:

To design and implement a **Symbol Table Manager** in C/C++ that accepts declaration statements, identifies valid identifiers, stores their attributes (symbol name, type, data size, and hexadecimal memory location), detects lexical errors and duplicate declarations, and displays the symbol table in a tabular format.

Algorithm:

1. Start the program.
2. Accept declaration statements from the keyboard or a text file.
3. Read each declaration statement.
4. Extract the data type of the declaration.
5. Separate the identifiers from the declaration.
6. For each identifier:
   - Check whether it follows the identifier naming rules.
   - If invalid, display a lexical error message.
   - If already present in the symbol table, display a duplicate declaration error.
   - Otherwise:
     - Determine the data size based on the data type.
     - Generate a hexadecimal memory location.
     - Store the symbol name, type, size, and memory location in the symbol table.
7. Repeat until all declarations are processed.
8. Display the complete symbol table in tabular format.

9. Stop the program.

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>

#define MAX 100

struct Symbol
{
   char name[30];
   char type[20];
   int size;
   unsigned int address;
};

struct Symbol table[MAX];
int count = 0;
unsigned int address = 0x1000;

int getSize(char type[])
{
   if(strcmp(type,"int")==0)
      return 4;
   else if(strcmp(type,"float")==0)
      return 4;
   else if(strcmp(type,"char")==0)
      return 1;
   else if(strcmp(type,"double")==0)
      return 8;
   else
      return 4;
}

int duplicate(char name[])
{
   for(int i=0;i<count;i++)
   {
      if(strcmp(table[i].name,name)==0)
         return 1;
   }
   return 0;
}

int validIdentifier(char id[])
{
   if(!(isalpha(id[0]) || id[0]=='_'))
      return 0;

   for(int i=1;id[i]!='\0';i++)
   {
      if(!(isalnum(id[i]) || id[i]=='_'))
         return 0;
   }
   return 1;
}
```

```
int main()
{
   int n;
   char line[100];

   printf("Enter number of declaration statements: ");
   scanf("%d",&n);
   getchar();

   for(int i=0;i<n;i++)
   {
      printf("Enter declaration %d: ",i+1);
      fgets(line,sizeof(line),stdin);

      line[strcspn(line,"\n")]='\0';

      char *token = strtok(line," ,;");
      if(token==NULL)
         continue;

      char type[20];
      strcpy(type,token);

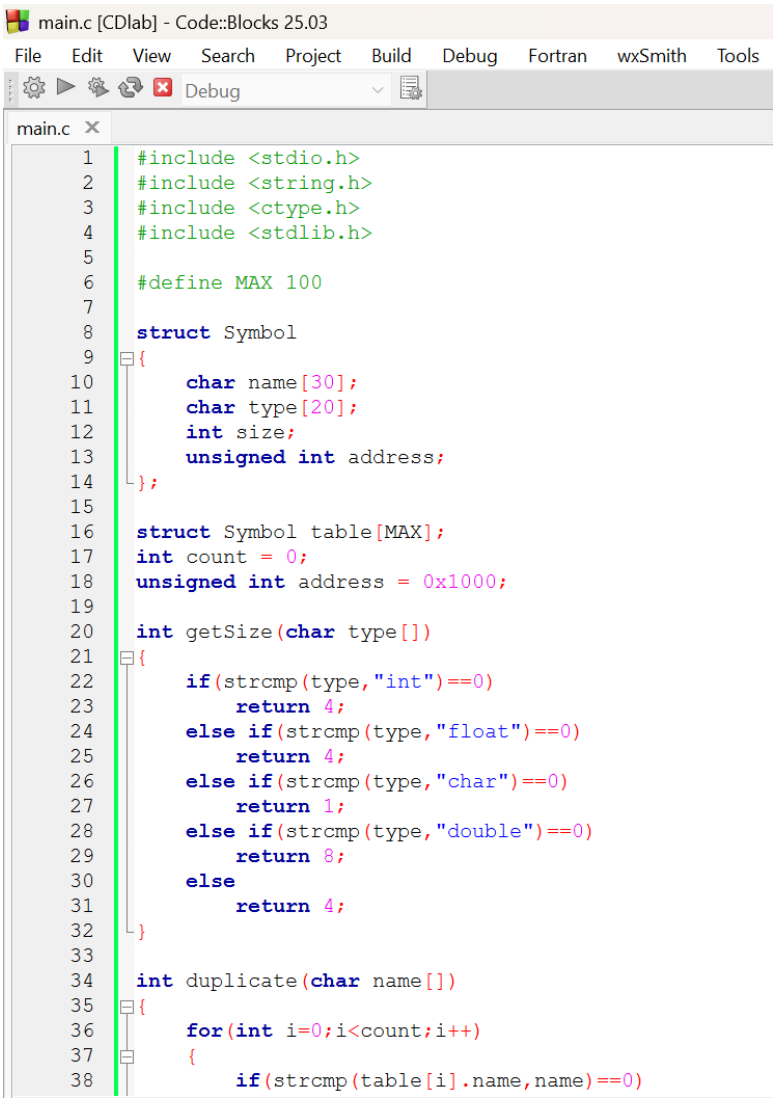
      while((token=strtok(NULL," ,;"))!=NULL)
      {
         if(!validIdentifier(token))
         {
            printf("Lexical Error: Invalid Identifier -> %s\n",token);
         }
         else if(duplicate(token))
         {
            printf("Error: Duplicate Declaration -> %s\n",token);
         }
         else
         {
            strcpy(table[count].name,token);
            strcpy(table[count].type,type);
            table[count].size=getSize(type);
            table[count].address=address;

            address+=table[count].size;
            count++;
         }
      }
   }

   printf("\n---------- SYMBOL TABLE ----------\n");
   printf("%-15s %-10s %-10s %-10s\n","Name","Type","Size","Address");

   for(int i=0;i<count;i++)
   {
      printf("%-15s %-10s %-10d 0x%X\n",
            table[i].name,
            table[i].type,
            table[i].size,
            table[i].address);
   }

   return 0;
}
```


main.c [CDlab] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools
Debug
main.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <stdlib.h>
#define MAX 100
struct Symbol
{
char name[30];
char type[20];
int size;
unsigned int address;
};
struct Symbol table[MAX];
int count = 0;
unsigned int address = 0x1000;
int getSize(char type[])
{
if(strcmp(type,"int")==0)
return 4;
else if(strcmp(type,"float")==0)
return 4;
else if(strcmp(type,"char")==0)
return 1;
else if(strcmp(type,"double")==0)
return 8;
else
return 4;
}
int duplicate(char name[])
{
for(int i=0;i<count;i++)
{
if(strcmp(table[i].name,name)==0)

main.c [CDlab] - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Setting

Debug

main.c

```
    {
        if(strcmp(table[i].name,name)==0)
            return 1;
    }
    return 0;
}

int validIdentifier(char id[])
{
    if(!(isalpha(id[0]) || id[0]=='_'))
        return 0;

    for(int i=1;id[i]!='\0';i++)
    {
        if(!(isalnum(id[i]) || id[i]=='_'))
            return 0;
    }
    return 1;
}

int main()
{
    int n;
    char line[100];

    printf("Enter number of declaration statements: ");
    scanf("%d",&n);
    getchar();

    for(int i=0;i<n;i++)
    {
        printf("Enter declaration %d: ",i+1);
        fgets(line,sizeof(line),stdin);

        line[strcspn(line,"\n")]='\0';

        char *token = strtok(line," ,;");
        if(token==NULL)
```

main.c [CDlab] - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Setti

Debug

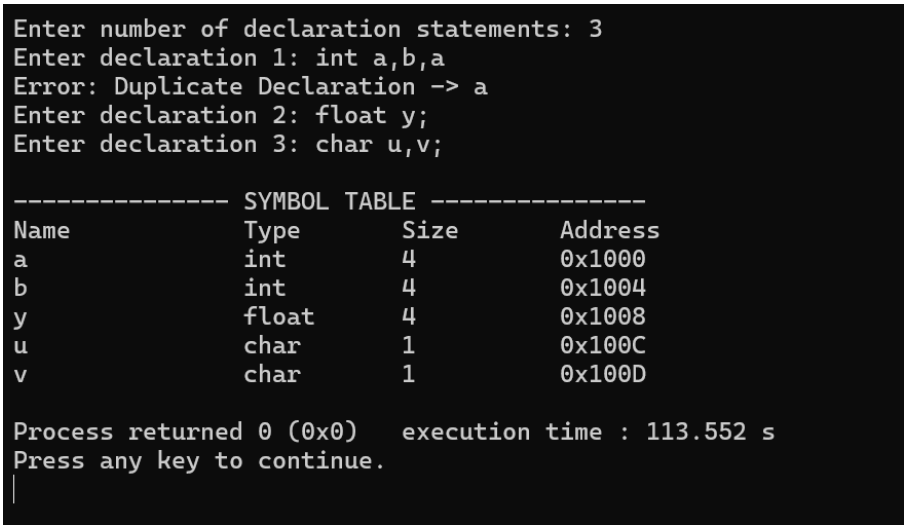
main.c

```
        if(token==NULL)
            continue;
        char type[20];
        strcpy(type,token);
        while((token=strtok(NULL," ,;"))!=NULL)
        {
            if(!validIdentifier(token))
            {
                printf("Lexical Error: Invalid Identifier -> %s\n",token);
            }
            else if(duplicate(token))
            {
                printf("Error: Duplicate Declaration -> %s\n",token);
            }
            else
            {
                strcpy(table[count].name,token);
                strcpy(table[count].type,type);
                table[count].size=getSize(type);
                table[count].address=address;

                address+=table[count].size;
                count++;
            }}}
    printf("\n--------------- SYMBOL TABLE ---------------\n");
    printf("%-15s %-10s %-10s %-10s\n","Name","Type","Size","Address");

    for(int i=0;i<count;i++)
    {
        printf("%-15s %-10s %-10d 0x%X\n",
               table[i].name,
               table[i].type,
               table[i].size,
               table[i].address);
    }
    return 0;
}
```

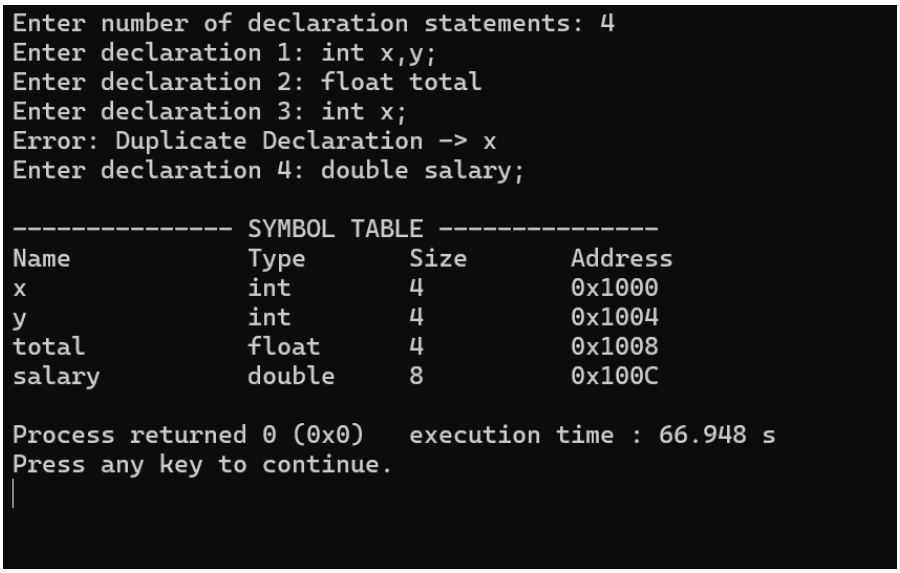
Input/Output Screenshots:



```
Enter number of declaration statements: 3
Enter declaration 1: int a,b,a
Error: Duplicate Declaration -> a
Enter declaration 2: float y;
Enter declaration 3: char u,v;

--------------- SYMBOL TABLE ---------------
Name            Type       Size       Address
a               int        4          0x1000
b               int        4          0x1004
y               float      4          0x1008
u               char       1          0x100C
v               char       1          0x100D

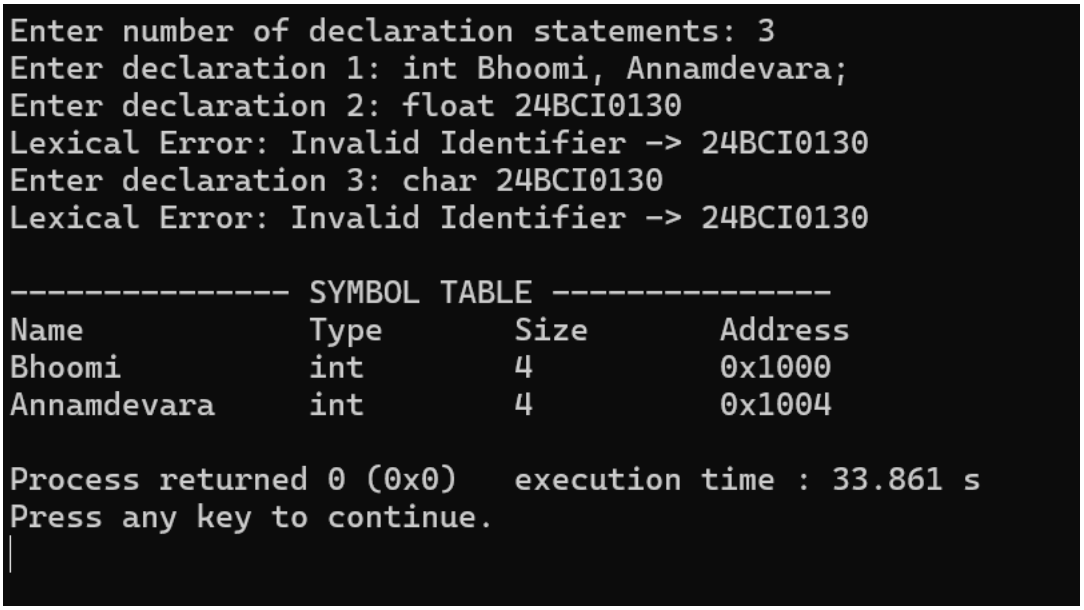
Process returned 0 (0x0)   execution time : 113.552 s
Press any key to continue.
```



```
Enter number of declaration statements: 4
Enter declaration 1: int x,y;
Enter declaration 2: float total
Enter declaration 3: int x;
Error: Duplicate Declaration -> x
Enter declaration 4: double salary;

--------------- SYMBOL TABLE ---------------
Name            Type       Size       Address
x               int        4          0x1000
y               int        4          0x1004
total           float      4          0x1008
salary          double     8          0x100C

Process returned 0 (0x0)   execution time : 66.948 s
Press any key to continue.
```



```
Enter number of declaration statements: 3
Enter declaration 1: int Bhoomi, Annamdevara;
Enter declaration 2: float 24BCI0130
Lexical Error: Invalid Identifier -> 24BCI0130
Enter declaration 3: char 24BCI0130
Lexical Error: Invalid Identifier -> 24BCI0130

--------------- SYMBOL TABLE ---------------
Name            Type       Size       Address
Bhoomi          int        4          0x1000
Annamdevara     int        4          0x1004

Process returned 0 (0x0)   execution time : 33.861 s
Press any key to continue.
```

```
Enter number of declaration statements: 5
Enter declaration 1: int num1,num2,result;
Enter declaration 2: float average,percentage;
Enter declaration 3:  char grade,section;
Enter declaration 4: double salary,bonus;
Enter declaration 5: int count;

--------------- SYMBOL TABLE ---------------
Name            Type       Size       Address
num1            int        4          0x1000
num2            int        4          0x1004
result          int        4          0x1008
average         float      4          0x100C
percentage      float      4          0x1010
grade           char       1          0x1014
section         char       1          0x1015
salary          double     8          0x1016
bonus           double     8          0x101E
count           int        4          0x1026

Process returned 0 (0x0)   execution time : 65.224 s
Press any key to continue.
```

```
Enter number of declaration statements: 6
Enter declaration 1: int a,b;
Enter declaration 2: float marks;
Enter declaration 3: int a;
Error: Duplicate Declaration -> a
Enter declaration 4: char 2grade;
Lexical Error: Invalid Identifier -> 2grade
Enter declaration 5: double salary$;
Lexical Error: Invalid Identifier -> salary$
Enter declaration 6: float _cgpa;

--------------- SYMBOL TABLE ---------------
Name            Type       Size       Address
a               int        4          0x1000
b               int        4          0x1004
marks           float      4          0x1008
_cgpa           float      4          0x100C

Process returned 0 (0x0)   execution time : 53.914 s
Press any key to continue.
```

Result:

Thus, the program to implement a Symbol Table Manager was successfully developed. It accepts declaration statements, identifies valid identifiers, stores symbol information (name, type, size, and hexadecimal memory location), detects lexical errors and duplicate declarations, and displays the symbol table successfully.

Design of a Simple Lexical Analyzer

Aim:

To implement a lexical analyzer in C that reads an input statement, identifies and classifies tokens such as keywords, identifiers, numbers, operators, delimiters, and reports lexical errors for invalid tokens.

Algorithm:

1. Start the program.
2. Read an input statement from the user.
3. Split the statement into individual lexemes (tokens).
4. For each token:
   - Check whether it is a keyword.
   - Else check whether it is an identifier.
   - Else check whether it is a number.
   - Else check whether it is an operator.
   - Else check whether it is a delimiter.
   - Otherwise report it as an invalid token (lexical error).
5. Display each lexeme along with its token type.
6. Stop the program.

Program:

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
char keywords[][10]={
"int","float","char","double",
"if","else","for","while","return","void"
};
int isKeyword(char str[])
{
   int i;
   for(i=0;i<10;i++)
   {
      if(strcmp(str,keywords[i])==0)
         return 1;
   }
   return 0;
}
int isOperator(char ch)
{
   char op[]="+-*/=<>";
   return strchr(op,ch)!=NULL;
}
int isDelimiter(char ch)
{
   char del[]="(){}[],;";
   return strchr(del,ch)!=NULL;
}
int isNumber(char str[])
{
   int i=0,dot=0;

   if(str[0]=='-' && str[1]!='\0')
      i=1;

   for(;str[i]!='\0';i++)
   {
      if(str[i]=='.')
      {
         dot++;
         if(dot>1)
            return 0;
      }
      else if(!isdigit(str[i]))
         return 0;
   }
   return 1;
```

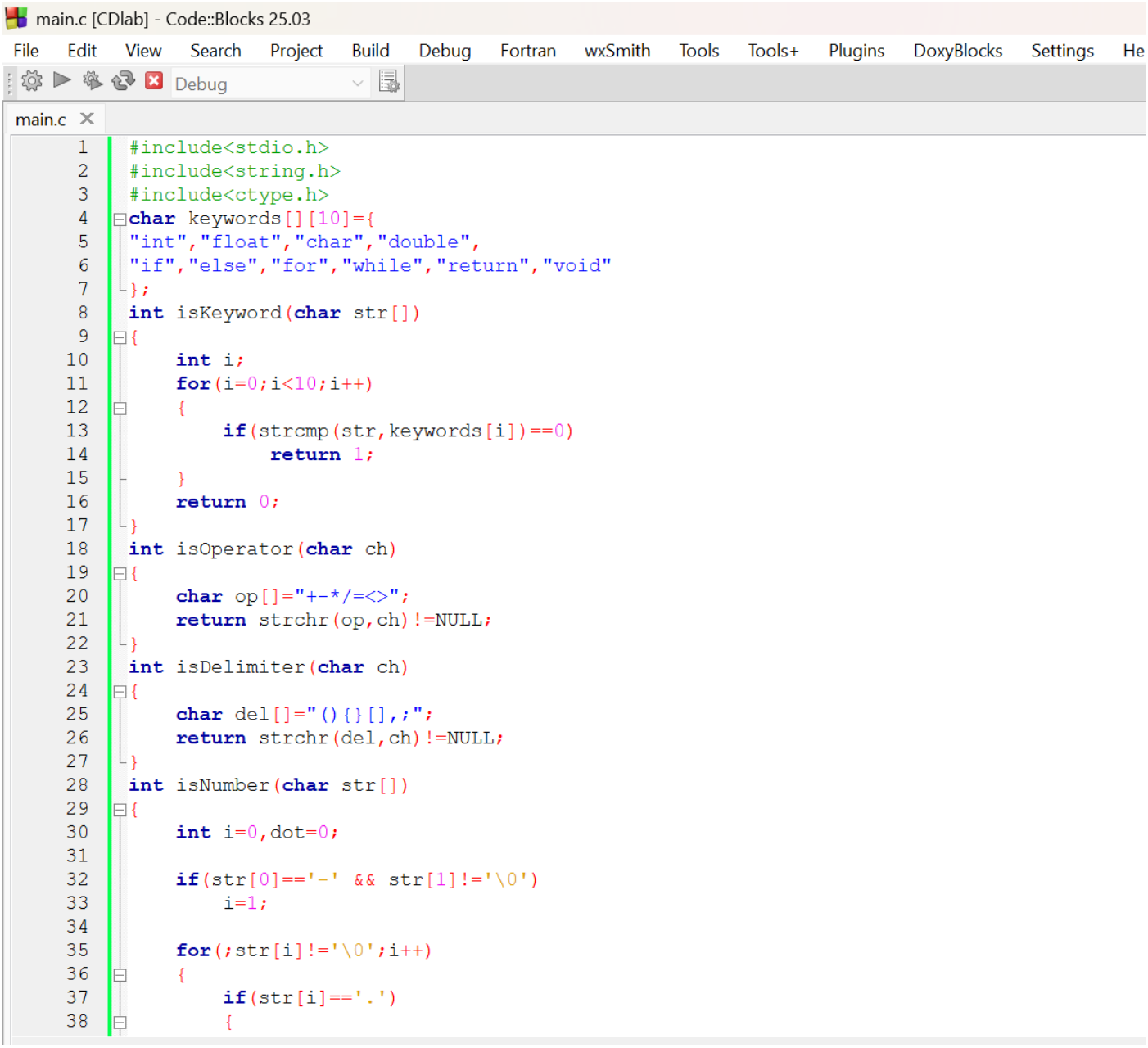
```
}
int isIdentifier(char str[])
{
   int i;
   if(!(isalpha(str[0])||str[0]=='_'))
      return 0;
   for(i=1;str[i]!='\0';i++)
   {
      if(!(isalnum(str[i])||str[i]=='_'))
         return 0;
   }
   return 1;
}
int main()
{
   char input[200];
   char token[50];
   int i=0,j=0;
   printf("Enter an expression:\n");
   fgets(input,sizeof(input),stdin);
   while(1)
   {
      if(input[i]==' '||input[i]=='\n'||input[i]=='\0')
      {
         if(j>0)
         {
            token[j]='\0';

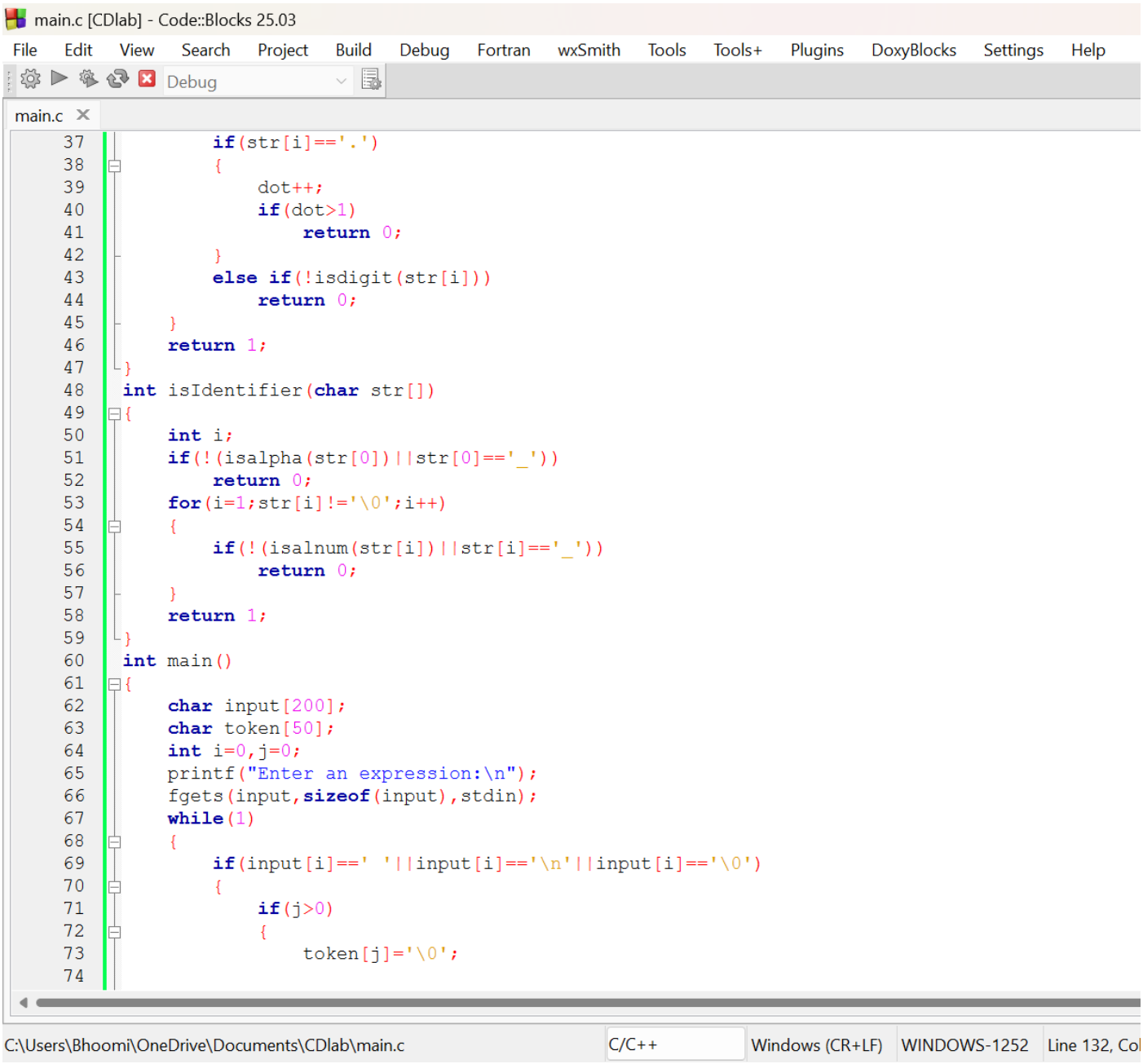
            if(isKeyword(token))
               printf("%s --> Keyword\n",token);

            else if(isIdentifier(token))
               printf("%s --> Identifier\n",token);
            else if(isNumber(token))
               printf("%s --> Number\n",token);

            else
               printf("%s --> Invalid Token\n",token);
            j=0;
         }
         if(input[i]=='\0')
            break;
      }
      else if(isOperator(input[i]))
      {
         if(j>0)
         {
```

```
            token[j]='\0';
            if(isKeyword(token))
               printf("%s --> Keyword\n",token);
            else if(isIdentifier(token))
               printf("%s --> Identifier\n",token);
            else if(isNumber(token))
               printf("%s --> Number\n",token);
            else
               printf("%s --> Invalid Token\n",token);
            j=0;
         }
         printf("%c --> Operator\n",input[i]);
      }
      else if(isDelimiter(input[i]))
      {
         if(j>0)
         {
            token[j]='\0';
            if(isKeyword(token))
               printf("%s --> Keyword\n",token);
            else if(isIdentifier(token))
               printf("%s --> Identifier\n",token);
            else if(isNumber(token))
               printf("%s --> Number\n",token);
            else
               printf("%s --> Invalid Token\n",token);
            j=0;
         }
         printf("%c --> Delimiter\n",input[i]);
      }
      else
      {
         token[j++]=input[i];
      }
      i++;
   }
   return 0;
}
```


main.c [CDlab] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings He
Debug
main.c
#include<stdio.h>
#include<string.h>
#include<ctype.h>
char keywords[][10]={
"int","float","char","double",
"if","else","for","while","return","void"
};
int isKeyword(char str[])
{
int i;
for(i=0;i<10;i++)
{
if(strcmp(str,keywords[i])==0)
return 1;
}
return 0;
}
int isOperator(char ch)
{
char op[]="+-*/=<>";
return strchr(op,ch)!=NULL;
}
int isDelimiter(char ch)
{
char del[]="(){}[],;";
return strchr(del,ch)!=NULL;
}
int isNumber(char str[])
{
int i=0,dot=0;
if(str[0]=='-' && str[1]!='\0')
i=1;
for(;str[i]!='\0';i++)
{
if(str[i]=='.')
{


main.c [CDlab] - Code::Blocks 25.03
File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help
Debug
main.c
37 if(str[i]=='.')
38 {
39 dot++;
40 if(dot>1)
41 return 0;
42 }
43 else if(!isdigit(str[i]))
44 return 0;
45 }
46 return 1;
47 }
48 int isIdentifier(char str[])
49 {
50 int i;
51 if(!(isalpha(str[0])||str[0]=='_'))
52 return 0;
53 for(i=1;str[i]!='\0';i++)
54 {
55 if(!(isalnum(str[i])||str[i]=='_'))
56 return 0;
57 }
58 return 1;
59 }
60 int main()
61 {
62 char input[200];
63 char token[50];
64 int i=0,j=0;
65 printf("Enter an expression:\n");
66 fgets(input,sizeof(input),stdin);
67 while(1)
68 {
69 if(input[i]==' '||input[i]=='\n'||input[i]=='\0')
70 {
71 if(j>0)
72 {
73 token[j]='\0';
74
C:\Users\Bhoomi\OneDrive\Documents\CDlab\main.c
C/C++
Windows (CR+LF)
WINDOWS-1252
Line 132, Col

main.c [CDlab] - Code::Blocks 25.03

File Edit View Search Project Build Debug Fortran wxSmith Tools Tools+ Plugins DoxyBlocks Settings Help

Debug

main.c

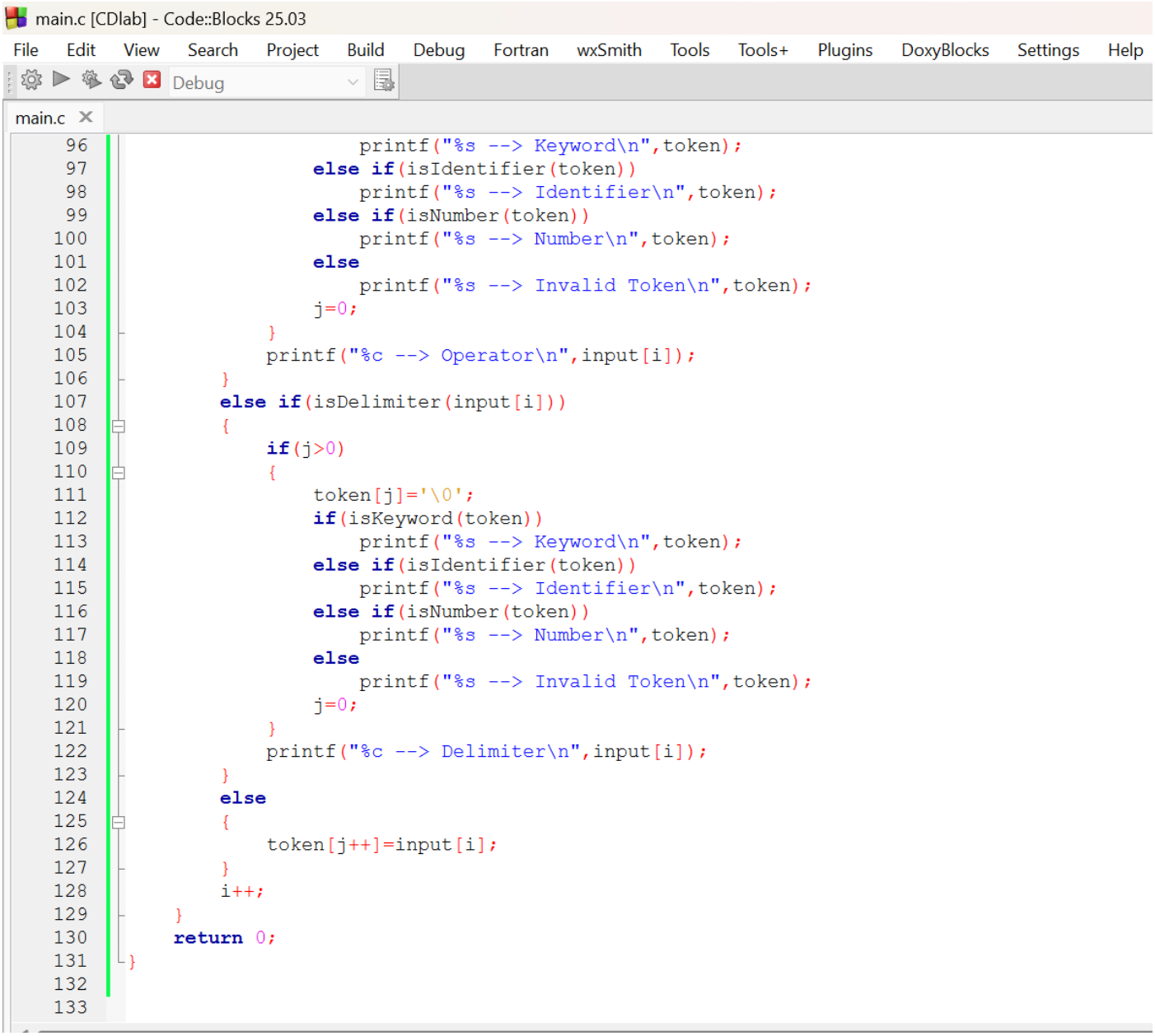
```
                token[j]='\0';

                if(isKeyword(token))
                    printf("%s --> Keyword\n",token);

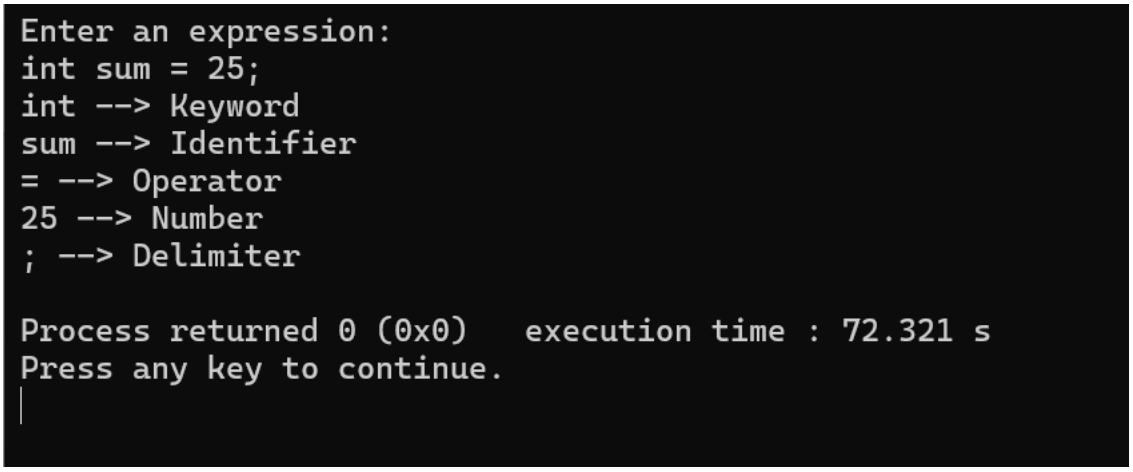
                else if(isIdentifier(token))
                    printf("%s --> Identifier\n",token);
                else if(isNumber(token))
                    printf("%s --> Number\n",token);

                else
                    printf("%s --> Invalid Token\n",token);
                j=0;
            }
            if(input[i]=='\0')
                break;
        }
        else if(isOperator(input[i]))
        {
            if(j>0)
            {
                token[j]='\0';
                if(isKeyword(token))
                    printf("%s --> Keyword\n",token);
                else if(isIdentifier(token))
                    printf("%s --> Identifier\n",token);
                else if(isNumber(token))
                    printf("%s --> Number\n",token);
                else
                    printf("%s --> Invalid Token\n",token);
                j=0;
            }
            printf("%c --> Operator\n",input[i]);
        }
        else if(isDelimiter(input[i]))
        {
            if(j>0)
            {
```

C:\Users\Bhoomi\OneDrive\Documents\CDlab\main.c | C/C++ | Windows (CR+LF) | WINDOWS-1252 | Line 132, Col 1, Pos



Input/Output Screenshots:



```
Enter an expression:
float avg = 85.5;
float --> Keyword
avg --> Identifier
= --> Operator
85.5 --> Number
; --> Delimiter

Process returned 0 (0x0)   execution time : 9.397 s
Press any key to continue.
```

```
Enter an expression:
if (count < 100)
if --> Keyword
( --> Delimiter
count --> Identifier
< --> Operator
100 --> Number
) --> Delimiter

Process returned 0 (0x0)   execution time : 1.563 s
Press any key to continue.
```

```
Enter an expression:
int 2marks = 50;
int --> Keyword
2marks --> Invalid Token
= --> Operator
50 --> Number
; --> Delimiter

Process returned 0 (0x0)   execution time : 1.938 s
Press any key to continue.
```

```
Enter an expression:
char grade$;
char --> Keyword
grade$ --> Invalid Token
; --> Delimiter

Process returned 0 (0x0)   execution time : 1.776 s
Press any key to continue.
```

```
Enter an expression:
bhoomi = 24BCI0130 + grade$$ + VIT
bhoomi --> Identifier
= --> Operator
24BCI0130 --> Invalid Token
+ --> Operator
grade$$ --> Invalid Token
+ --> Operator
VIT --> Identifier

Process returned 0 (0x0)   execution time : 32.162 s
Press any key to continue.
```

RESULT:

Thus, the lexical analyzer was successfully implemented in C. The program correctly identifies keywords, identifiers, numbers, operators, delimiters, and reports invalid tokens (lexical errors).